bupty：纯 Bash 的北邮一站式信息查询终端

Linux 课程作业——架构设计、实现与测试

2025211681

2026 年 10 月

目录

1. 动机
2. 功能
3. 架构
4. 设计选择
5. 测试
6. 总结

动机：多平台割裂的日常痛点

- 北邮学生日常要登录多个相互独立的信息系统：
  - 移动教务（课表、空教室、成绩、考试）
  - 教学云 UCloud（课程作业与 DDL）
  - 信息门户 / 公共接口（班车、节假日、竞赛 DDL）
  - 图书馆研修间预约系统
- 频繁在各平台之间切换、重复登录，信息分散、易遗漏（尤其作业 DDL 与抢约）。

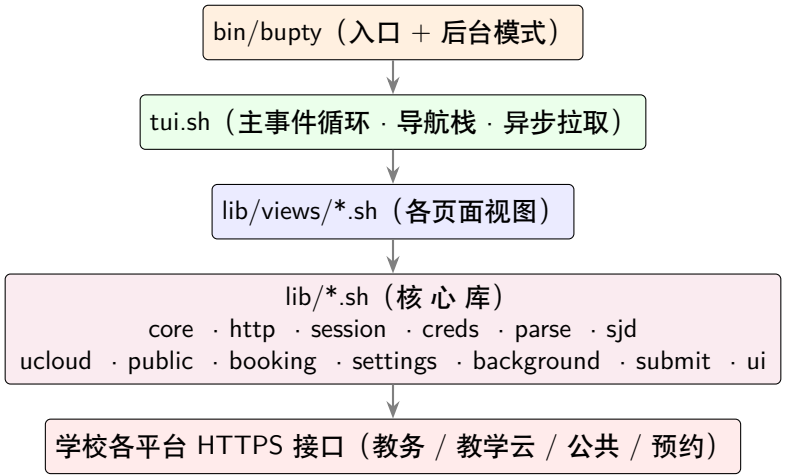
动机：为什么重写、为什么用 Bash

- 参考项目 `where_to_study` 功能完整，但 CLI 需要传大量参数，操作不直观。
- 我们希望提供一个 **TUI（终端图形界面）**：方向键选择、Enter 进入、Esc 返回，交互符合直觉。
- 课程要求用 **Bash** 实现：本项目全部由 Bash + 少量系统工具（`curl`、`jq`）构成，无额外运行时依赖。

功能总览

- **作业 DDL**：教学云作业聚合、截止时间排序、紧急程度标注、在线提交。
- **课表**：周节次网格，可切周 / 切星期。
- **空教室**：双校区，按楼 / 楼层 / 时段筛选，可隐藏教室。
- **成绩 / 考试**：学期成绩、平均绩点、考试安排。
- **公共查询**：班车时刻、法定节假日、竞赛活动 DDL。
- **研修间预约**：浏览房间 / 空闲时段、预约、取消，以及「自动抢约」。
- **后台守护**：开机自启动 + 紧急 DDL 弹窗通知 + 抢约自动下单。
- **记住密码**：凭据本地安全保存，会话令牌仅存内存。

整体架构：分层与模块


bin/bupty（入口 + 后台模式）
tui.sh（主事件循环 · 导航栈 · 异步拉取）
lib/views/*.sh（各页面视图）
lib/*.sh（核 心 库）
core · http · session · creds · parse · sjd
ucloud · public · booking · settings · background · submit · ui
学校各平台 HTTPS 接口（教务 / 教学云 / 公共 / 预约）

数据流：JSON 贯穿始终

- 各网络模块统一输出 **JSON 字符串**（用 `jq` 生成 / 消费），模块间不拼接脆弱的分隔符。
- 视图层先做一次 `jq` **预计算**（TSV 行缓存），渲染循环内用纯 Bash 读取，不再反复调用 `jq`。
- 典型链路：网络模块 $\xrightarrow{\texttt{JSON}}$ 视图预计算 $\xrightarrow{\texttt{TSV}}$ 逐行渲染。

异步加载：进入页面即时显示、后台拉取

- 首次进入某页面时，后台子进程拉取数据（`_async_start`），页面先显示「正在加载…」。
- 主循环在每个超时点轮询（`_async_poll`），完成后回调 `apply_<view>` 并重绘。
- 渲染用「帧缓冲」：整帧先写入临时文件再一次性输出，避免逐行重绘闪烁；满屏时页脚末行不带换行，避免顶出页眉。

设计选择（一）：Bash 的模块化

- 不写「巨型 shell」，而是按职责拆分为多个 `source` 加载的模块文件。
- 命名约定：纯解析函数（`lib/parse.sh`）**不做网络**，便于离线单测；网络函数返回 `0/1` 并把结果写 stdout、错误写全局 `BUPTY_ERR`。
- 视图 = `render_<name>()` + `handle_<name>_key()`，与主循环解耦。

设计选择（二）：渲染性能

- **消除子进程 fork**：渲染热循环里用无 fork 的 `_core_fit_compute` 计算显示宽度，用参数展开切分 TSV，避免 `$(...)` 与 herestring。
- **单趟 O(n) 截断**：中文按 2 列宽截断文本，避免逐字符回退造成的 O(n^2) 卡顿。
- **结果缓存**：凭据 / 活动过滤 / 班车显示行只算一次；课表、班车等页从约 200ms 降到约 10–30ms/键。

设计选择（三）：安全

- 凭据目录 0700、文件 0600，写入时拒绝符号链接；密码永不打印、不进日志。
- 教务 / 教学云 / 预约的令牌仅存内存，过期自动重登一次；预约使用 CAS 七步认证，全程同一 cookie jar、一次性 execution 用后即弃。
- 会话退出时清理 cookie jar。

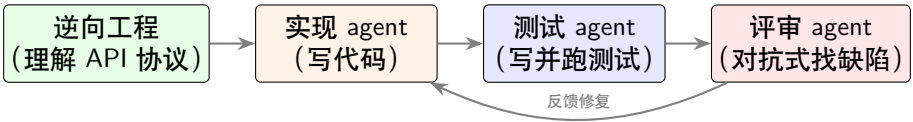
设计选择（四）：可配置与后台

- 设置持久化到 `settings.json`（临时文件 + `mv` 原子写）。
- `--background` 后台模式：单实例锁，循环拉作业并对「紧急 DDL」弹窗；自动抢约只抢「明天及以后」，永久性错误丢弃、临时错误限次重试。

测试方法

- **单元测试**（`tests/unit_*.sh`）：离线、基于 fixture JSON，断言解析 / 组装 / 过滤等纯函数。
- **渲染快照**：渲染各视图并断言关键文本（去 ANSI），验证界面文案。
- **多窗口尺寸自检**：在 80/100/120/60/140 列下断言不溢出、页眉在、无横向换行。
- **真实网络集成测试**（只读 + 授权）：登录、拉取、预约 → 取消，验证协议与实现一致。
- **对抗式评审**：实现 / 测试 / 评审由不同 agent 分别完成，不共享上下文。

多代理开发流程



每一轮：一个 agent 实现、一个写测试、一个对抗式评审，彼此不共用上下文，避免「自我蒙混」。

总结

- 用纯 Bash 实现了一个功能完整、交互友好的北邮信息查询 **TUI**。
- 核心经验：模块化拆分、JSON 作为数据契约、异步加载 + 帧缓冲、渲染热循环去 fork、O(n) 宽度计算。
- 通过「单元测试 + 渲染快照 + 多尺寸自检 + 真实网络集成 + 对抗式评审」保证了正确性与界面质量。
- 网络协议（教务 / 教学云 / 公共 / 预约）均经真实请求逐条逆向核对并文档化。

谢谢！